HÖGSKOLAN DALARNA

Assignment: Linux lab

Names
Ali Adrian Nasrat, Anna Backolars

DU-Identity
h22alian, h23annba

Date
13/10/25

Course code
GIK2NV

Linux Lab

1 Manual implementation

The first part of the laboratory focused on configuring a Linux system for three newly established departments, **Engineering**, **Sales**, and **HR**, through the manual creation of directories, groups, users, and access controls. The purpose was to construct a secure, well-structured environment that mirrored how departmental segregation and privilege management are applied in real corporate infrastructures.

To begin, departmental directories were created under the root of the file system using the command

cd /

sudo mkdir Engineering Sales HR.

A subsequent listing with ls -l verified that the directories had been correctly established and owned by the root user (Figure 1). These directories serve as isolated workspaces for each department.

```
ubuntu@ubuntu:/$ sudo mkdir Engineering Sales HR
ubuntu@ubuntu:/$ ls -l
total 88
drwxr-xr-x   2 root root  4096 Oct 13 17:44 Engineering
drwxr-xr-x   2 root root  4096 Oct 13 17:44 HR
drwxr-xr-x   2 root root  4096 Oct 13 17:44 Sales
```

Figure 1. Verification of the department directories created at the root level.

Next, groups were created to correspond with each department using sudo groupadd engineering, sudo groupadd sales, and sudo groupadd hr. The getent group command confirmed that all three groups were successfully added to the system's group database (Figure 2). Group creation was a crucial preparatory step, as these entities later defined collective permissions for departmental members.

```
ubuntu@ubuntu:/$ sudo groupadd engineering
ubuntu@ubuntu:/$ sudo groupadd sales
ubuntu@ubuntu:/$ sudo groupadd hr
ubuntu@ubuntu:/$ getent group engineering sales hr
engineering:x:1006:
sales:x:1007:
hr:x:1008:
```

Figure 2. Confirmation of group creation using the getent group command.

Following group creation, administrative users were introduced for each department. These users, engadmin, salesadmin, and hradmin, were assigned Bash as their login shell and their departmental group as the primary group. Passwords were set using passwd, ensuring that each account was immediately functional (Figure 3). Verification with id and getent passwd confirmed that each administrative user was properly linked to its respective group and configured with the correct shell.

```
ubuntu@ubuntu:/$ sudo useradd -m -s /bin/bash -g engineering engadmin
ubuntu@ubuntu:/$ sudo useradd -m -s /bin/bash -g sales salesadmin
ubuntu@ubuntu:/$ sudo useradd -m -s /bin/bash -g hr hradmin
ubuntu@ubuntu:/$ sudo passwd engadmin
New password:
Retype new password:
passwd: password updated successfully
ubuntu@ubuntu:/$ sudo passwd salesadmin
New password:
Retype new password:
passwd: password updated successfully
ubuntu@ubuntu:/$ sudo passwd hradmin
New password:
Retype new password:
passwd: password updated successfully
ubuntu@ubuntu:/$ getent passwd engadmin salesadmin hradmin
engadmin:x:1001:1006::/home/engadmin:/bin/bash
salesadmin:x:1002:1007::/home/salesadmin:/bin/bash
hradmin:x:1003:1008::/home/hradmin:/bin/bash
ubuntu@ubuntu:/$ id engadmin
uid=1001(engadmin) gid=1006(engineering) groups=1006(engineering)
ubuntu@ubuntu:/$ id salesadmin
uid=1002(salesadmin) gid=1007(sales) groups=1007(sales)
ubuntu@ubuntu:/$ id hradmin
uid=1003(hradmin) gid=1008(hr) groups=1008(hr)
```

Figure 3. Creation and verification of administrative users for each department.

To populate the departments, two additional users were created per group with the same configuration parameters, a Bash shell and the corresponding department group as their primary group. These commands are displayed in Figure 4. The accounts were successfully established, as demonstrated in Figure 5, where the id command output verifies that each user belongs exclusively to the correct departmental group.

```
ubuntu@ubuntu:/$ sudo useradd -m -s /bin/bash -g engineering enguser1
ubuntu@ubuntu:/$ sudo useradd -m -s /bin/bash -g engineering enguser2
ubuntu@ubuntu:/$ sudo useradd -m -s /bin/bash -g sales salesuser1
ubuntu@ubuntu:/$ sudo useradd -m -s /bin/bash -g sales salesuser2
ubuntu@ubuntu:/$ sudo useradd -m -s /bin/bash -g hr hruser1
ubuntu@ubuntu:/$ sudo useradd -m -s /bin/bash -g hr hruser2
```

Figure 4. Creation of two additional users per department using useradd with Bash shell.

```
ubuntu@ubuntu:/$ id enguser1 enguser2 salesuser1 salesuser2 hruser1 hruser2
uid=1004(enguser1) gid=1006(engineering) groups=1006(engineering)
uid=1005(enguser2) gid=1006(engineering) groups=1006(engineering)
uid=1006(salesuser1) gid=1007(sales) groups=1007(sales)
uid=1007(salesuser2) gid=1007(sales) groups=1007(sales)
uid=1008(hruser1) gid=1008(hr) groups=1008(hr)
uid=1009(hruser2) gid=1008(hr) groups=1008(hr)
```

Figure 5. Verification of departmental user accounts and their respective group memberships.

After creating all users, directory ownership and access control lists were modified to ensure proper data segregation. Using the chown command, ownership of each directory was transferred to the department administrator and group, and full access rights were granted to both entities via chmod 770. The results in Figure 6 confirm that the ownership and permissions were correctly applied. To further enhance security, the *sticky bit* (chmod +t) was added to each directory, ensuring that only the creator of a file within a directory may delete it. The updated directory attributes are displayed in Figure 7, showing the T flag that denotes the sticky bit.

```
ubuntu@ubuntu:/$ ls -ld /Engineering /Sales /HR
drwxrwx--- 2 engadmin   engineering 4096 Oct 13 17:44 /Engineering
drwxrwx--- 2 hradmin    hr          4096 Oct 13 17:44 /HR
drwxrwx--- 2 salesadmin sales       4096 Oct 13 17:44 /Sales
```

Figure 6. Directory ownership transferred to departmental administrators with full group access rights.

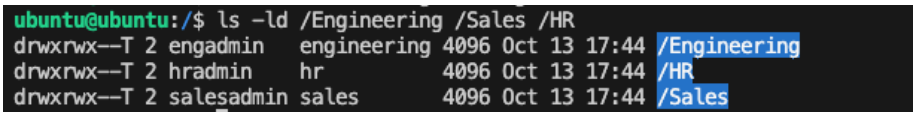

Figure 7. Sticky bit applied to departmental directories to restrict file deletion to file owners.

To confirm that users were properly associated with their departmental groups, the getent group command was executed once more (Figure 8). The output clearly lists all users belonging to their corresponding groups, providing evidence that group assignments were implemented correctly and consistently across the system.

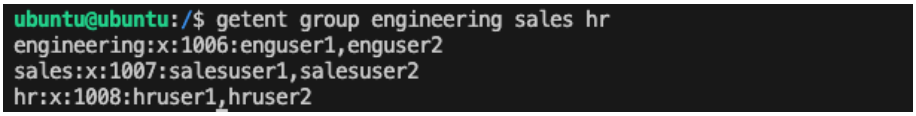

Figure 8. Verification of group memberships for Engineering, Sales, and HR users.

The final configuration step involved creating a departmental document within each directory. The file confidential.txt was first generated using touch and then edited in vi to include the single line of text *"This file contains confidential information for the department."* as shown in Figure 9. Ownership of each file was set to the corresponding administrator and group using chown, and permissions were restricted with chmod 640, granting read/write access to the administrator, read-only access to group members, and no access to others. The ls -l output in Figure 10 verifies that these permissions were correctly applied across all departmental files.

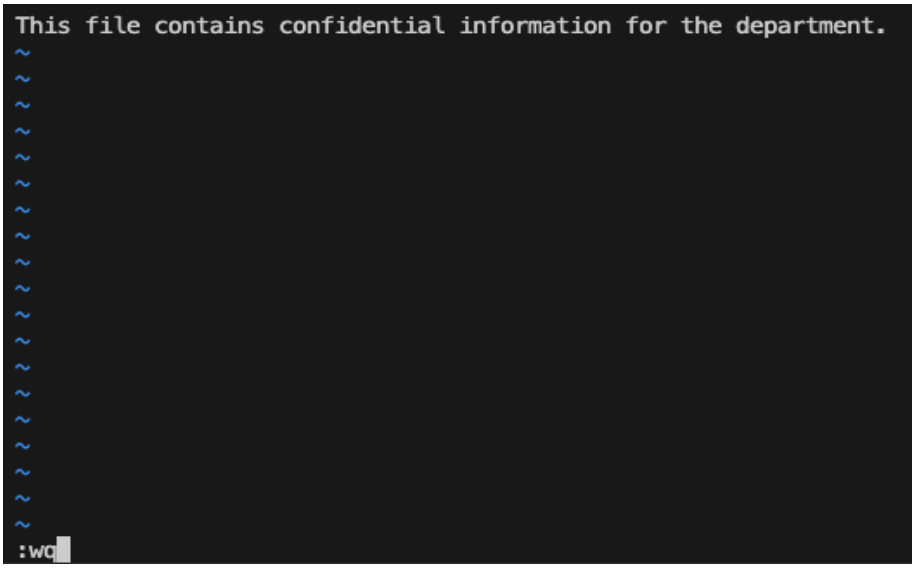

Figure 9. Editing of the confidential departmental file in vi.

```
ubuntu@ubuntu:/$ sudo ls -l /Engineering/confidential.txt /Sales/confidential.txt /HR/confid
ential.txt
-rw-r----- 1 engadmin   engineering 64 Oct 13 18:32 /Engineering/confidential.txt
-rw-r----- 1 hradmin    hr          64 Oct 13 18:33 /HR/confidential.txt
-rw-r----- 1 salesadmin sales       64 Oct 13 18:32 /Sales/confidential.txt
```

Figure 10. Verification of file ownership and restricted permissions for confidential.txt.

To confirm that the file content was accurately written and accessible according to the specified permissions, the command sudo cat /Engineering/confidential.txt was executed. The output, displayed in Figure 11, shows the text string contained in the document and demonstrates that the file is readable within its department while remaining protected from modification by non-administrative users.

```
ubuntu@ubuntu:/$ sudo cat /Engineering/confidential.txt
This file contains confidential information for the department.
```

Figure 11. Display of the confidential file's contents confirming readability and data integrity.

Through these steps, the manual configuration achieved a secure departmental structure in which administrative users possess full control, departmental users retain collaborative read and write access within their areas, and cross-departmental access is fully restricted. The system therefore meets both the organizational and security objectives stated in the case scenario.

2 Automated script implementation

The second part of the laboratory consisted of developing and executing a Bash automation script designed to replicate all the user, group, and directory management tasks performed manually in Part 1. The objective of this phase was to demonstrate how administrative procedures can be automated in Linux to ensure accuracy, security, and scalability in system configuration.

The script, named user-management.sh, was written to sequentially execute the nine administrative steps described in the assignment instructions. These include the creation of groups, user accounts, directory structures, and permission management according to enterprise-level security principles. Figure 1 shows the script file as opened in the terminal, located within the lab3 directory. The script contains several well-structured functions, including create_group, prompt_username_unique, prompt_password_confirm, create_user_in_group, and setup_department_dir, each of which performs a specific administrative task.

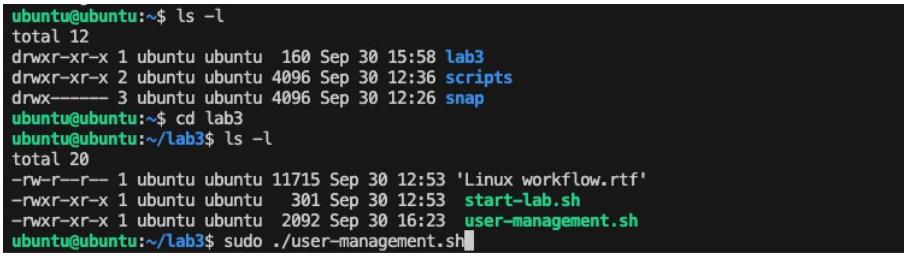

Figure 1. Opening the Bash automation script user-management.sh in the terminal.

When the script is executed using superuser privileges (sudo ./user-management.sh), it first prompts the administrator to enter a new group name, as shown in Figure 2. This interactive input ensures that the script can dynamically adapt to different departmental or team configurations. The design includes input validation, preventing duplicate group names by checking against existing system entries using the getent group command.

Figure 2. Execution of the script prompting for the creation of a new group.

After the group name is entered and verified, the script confirms the creation of the group in real time. Figure 3 displays the console output confirming that the group *developers* has been successfully created.

Figure 3. Confirmation of successful creation of the new group "developers."

Next, the script requests input for a new username and password, following a similar validation procedure to ensure both uniqueness and correctness. Figure 4 illustrates the process in which the user devuser1 is created, assigned the Bash login shell, and added to the group *developers*. This step implements secure password confirmation by prompting for two matching entries, minimizing the likelihood of misconfiguration.

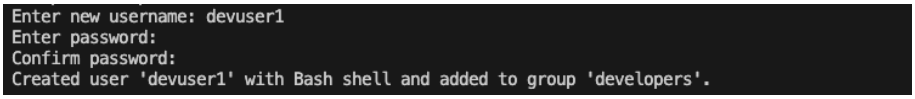

Figure 4. Creation of a new user with secure password confirmation and group assignment.

Upon creation of the user account, the script automatically generates a departmental directory corresponding to the username. This directory is established at the root level and configured with permission mode 3770, combining full access for the owner and group with the sticky bit to protect files from accidental deletion. Figure 5 demonstrates the verification of this directory's existence and attributes through the ls -ld command.

Figure 5. Verification of directory creation and permission mode configuration for the new user.

Once the directory structure is in place, the script continues to the verification phase, where it retrieves and displays information about both the user and their assigned group. Figure 6 shows the outputs of getent group and id, confirming that devuser1 is properly associated with the *developers* group and that the group itself has been successfully registered within the system database.

```
ubuntu@ubuntu:~/lab3$ getent group developers
developers:x:1009:
ubuntu@ubuntu:~/lab3$ id devuser1
uid=1010(devuser1) gid=1009(developers) groups=1009(developers)
```

Figure 6. Validation of group registration and user membership using getent group and id.

To ensure that everything is correct, the script concludes by printing a comprehensive verification summary, as seen in Figure 7. This includes directory ownership, permission details, and user and group metadata retrieved through stat and id. Such verification is crucial in automated administration, as it provides immediate visual confirmation that each task has executed as expected.

```
==== Department Directory Verification ====
Directory: /devuser1
drwxrws--T 2 devuser1 developers 4096 Oct 13 19:21 /devuser1
Mode: drwxrws--T (3770) Owner: devuser1 Group: developers
User info:
uid=1010(devuser1) gid=1009(developers) groups=1009(developers)
Group info:
developers:x:1009:
```

Figure 7. Automated verification summary confirming directory ownership and permission consistency.

The automation logic embedded in the script effectively encapsulates all the tasks listed under items (a) through (i) in the assignment instructions. It creates groups, validates input, generates users with secure passwords, constructs departmental directories, applies precise ownership and permission settings, and prints verification information. Throughout the process, the script safeguards against duplicate entries and enforces a uniform permission policy that mirrors the manual procedure from Part 1.

By automating this configuration process, administrative efficiency is significantly improved. The script eliminates repetitive manual input, ensures consistent security configurations, and provides reproducible system states. The clear modular design also allows future administrators to expand or modify departmental structures without rewriting core logic. Overall, this implementation exemplifies how routine system administration tasks can be transformed into robust, auditable, and maintainable automation routines.